

Exporting data from within a Promise response requires handling the asynchronous nature of the data, as the code exporting the data usually executes before the Promise resolves. [🔗](#)

Here are the best practices for exporting data derived from a Promise:

1. Export the Promise Itself (Recommended) [🔗](#)

Instead of exporting the *value* (which is undefined at first), export the Promise object itself. The consuming module can then use `.then()` or `await` to handle the data once it is available. [🔗 Stack Overflow +4](#)

Module A (data.js):

javascript

```
const myPromise = new Promise((resolve) => {
  setTimeout(() => resolve("Data loaded"), 1000);
});

// Export the Promise
module.exports = myPromise;
```

Module B (main.js):

javascript

```
const dataPromise = require('./data');

dataPromise.then(data => {
  console.log(data); // "Data Loaded"
});
```

2. Use `async/await` in ES Modules (Top-level await)

If using ECMAScript Modules (ESM) in Node.js or modern browser environments, you can use `await` at the top level of your file to pause execution until the promise resolves before exporting. [🔗 Stack Overflow +1](#)

Module A (data.js):

javascript

```
const data = await fetch('https://api.example.com').then(r => r.json());
export { data };
```

3. Using a Callback/Setter Function

You can export a function that acts as a setter to fill a local variable once the promise resolves. [The freeCodeCamp Forum +1](#)

Module A (data.js):

javascript

```
let data;
async function loadData() {
  data = await fetch('...').then(r => r.json());
}
loadData();
export { data };
```

4. Direct Modification (module.exports)

You can use a `.then()` block to directly modify the `module.exports` object once the data is ready. However, this is risky if the consumer tries to use the module synchronously. [Stack Overflow +1](#)

Module A (data.js):

javascript

```
const dataObj = {};
module.exports = dataObj; // Export empty object

new Promise(resolve => setTimeout(() => resolve('Ready'), 1000))
  .then(res => {
    dataObj.data = res; // Populate later
  });
```

Key Considerations

- **Asynchronous Nature:** You cannot synchronously return values from a Promise. The consuming code *must* wait.

- **Error Handling:** Always include a `.catch()` block when consuming promises to handle potential errors.
- **Event Loop:** Remember that Promise tasks are placed in the microtask queue, meaning they execute after synchronous code completes.  YouTube +4